

Machine Learning Notes

Preliminary Introduction to Deep Learning

Vitor N. Cabral de Oliveira

vnco@cin.ufpe.br

August 1, 2026

This notebook serves as a comprehensive repository for my Machine Learning studies, with a primary focus on Deep Learning. Key topics covered include:

- Artificial Neural Networks (ANNs) – Fundamentals and architectures with mathematical explanation.

Contents

Artificial Neural Networks

Building on the probabilistic and statistical learning foundations, we now introduce artificial neural networks (ANNs) – a flexible class of models inspired by biological neurons. ANNs can approximate any continuous function given sufficient capacity, which is why they form the backbone of modern deep learning. We begin with the simplest unit, the perceptron, then progress to multilayer networks trained via the backpropagation algorithm.

The Perceptron

Nearly every modern neural network, from simple classifiers to sprawling deep architectures, traces its origins to a simple computational unit called the **perceptron**¹. Developed by Frank Rosenblatt in 1947 (inspired by McCulloch and Pitts, 1943), the perceptron is a binary decision-maker. It takes a set of real-valued inputs, weighs their importance, and makes a crisp, threshold-based choice. Imagine a perceptron as a tiny gatekeeper: it evaluates evidence (inputs), applies its own biases and preferences (weights), and delivers a verdict – either 1 (“yes”) or 0 (“no”).

Mathematical Definition Given inputs $x_1, x_2, \dots, x_n \in \mathbb{R}$, the perceptron’s output is governed by two steps:

1. **Weighted sum:** compute an affine transformation $z = w_0 + \sum_{i=1}^n w_i x_i$, where w_i are real-valued **weights** and w_0 is the **bias**.
2. **Thresholding:** apply the Heaviside step function $\phi(z) = \mathbf{1}_{\{z > 0\}}$, yielding $o = \phi(z)$.

By introducing a dummy input $x_0 = 1$, we can absorb the bias into the weight vector: let $\mathbf{w} = (w_0, w_1, \dots, w_n)^\top$ and $\mathbf{x} = (1, x_1, \dots, x_n)^\top$. Then $z = \mathbf{w} \cdot \mathbf{x}$ and $o = \mathbf{1}_{\{\mathbf{w} \cdot \mathbf{x} > 0\}}$.

Geometric Interpretation The equation $\mathbf{w} \cdot \mathbf{x} = 0$ defines a hyperplane in $\mathbb{R}^{n+1}$. This decision boundary partitions the input space into two half-spaces:

- $\mathbf{w} \cdot \mathbf{x} > 0$ corresponds to output +1 (positive class),
- $\mathbf{w} \cdot \mathbf{x} < 0$ corresponds to output 0 (negative class).

For two-dimensional inputs (x_1, x_2) , the boundary is a line: $w_0 + w_1 x_1 + w_2 x_2 = 0$. The bias w_0 controls the offset from the origin, while the ratio w_1/w_2 determines the slope. The weight vector $\mathbf{w}$ is always normal (perpendicular) to this line and points toward the positive region.

¹ While most neural networks rely on perceptron-like units, alternative paradigms exist, such as Hebbian learning and the recently revived Kolmogorov–Arnold Networks (KANs).

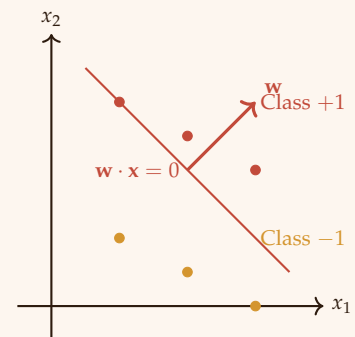


Figure 1: Geometric interpretation of the perceptron: the weight vector $\mathbf{w}$ is orthogonal to the decision hyperplane and points toward the positive class.

The XOR Limitation The perceptron can represent all primitive Boolean functions (AND, OR, NAND, NOR) because they are linearly separable. However, it fails for the XOR function: $x_1 \oplus x_2 = 1$ iff $x_1 \neq x_2$. Figure ?? shows that no single line can separate the points (0,0) and (1,1) from (1,0) and (0,1). This limitation, highlighted by Minsky and Papert in 1969, led to the first “AI winter” and motivated the development of multilayer networks.

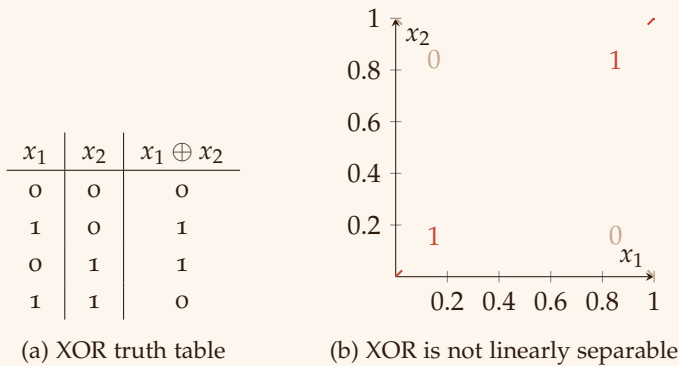


Figure 2: The XOR problem: no single straight line can separate the two classes.

Learning Rules for a Single Unit

Perceptron Training Rule

One way to find an acceptable weight vector is to start with random weights and iteratively update them whenever the perceptron misclassifies an example. The perceptron rule modifies weight w_i (associated with input x_i) by

$$\Delta w_i = \eta (t - o) x_i,$$

where t is the target output (0 or 1), o the perceptron’s output, and $\eta \in (0, 1)$ the **learning rate**. This rule converges if the data are linearly separable; otherwise it may oscillate.

Delta Rule and Gradient Descent

For non-separable problems, we need a rule that minimises a continuous error function. Consider a **linear unit** $o = \mathbf{w} \cdot \mathbf{x}$ (no threshold). Define the sum of squared errors over a training set $\mathcal{D} = \{(\mathbf{x}_d, t_d)\}_{d=1}^N$:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{d=1}^N (t_d - o_d)^2.$$

The factor $\frac{1}{2}$ simplifies differentiation. The gradient of E with respect to $\mathbf{w}$ is

$$\nabla E(\mathbf{w}) = \sum_{d=1}^N (o_d - t_d) \mathbf{x}_d,$$

so the gradient descent update (batch mode) becomes

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E(\mathbf{w}) = \mathbf{w} + \eta \sum_{d=1}^N (t_d - o_d) \mathbf{x}_d.$$

Remark 0.1 (Connection to Maximum Likelihood Estimation)

Assume a regression model $y = \mathbf{w}^T \mathbf{x} + \varepsilon$, with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. The likelihood of the data is $\prod_{d=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_d - \mathbf{w}^T \mathbf{x}_d)^2}{2\sigma^2}\right)$. Taking negative logs and discarding constants gives $\frac{1}{2\sigma^2} \sum_d (y_d - \mathbf{w}^T \mathbf{x}_d)^2$. Minimising this is exactly the delta rule (scaled by $1/\sigma^2$). Hence, minimising the squared error is equivalent to maximum likelihood estimation under a Gaussian noise model.

Stochastic Gradient Descent (SGD)

For large N , computing the full gradient every iteration is expensive. SGD approximates it using a single randomly chosen example (or a small mini-batch):

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell_i(\mathbf{w}),$$

where $\nabla \ell_i$ is the gradient of the loss for the i -th example. The stochasticity introduces noise that can help escape shallow local minima, and the algorithm is the workhorse of deep learning.

Activation Functions

To build multilayer networks, we need nonlinear, differentiable activation functions. Without nonlinearity, stacking linear layers would still produce a linear function. Common choices include:

Definition 0.1 (Sigmoid)

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1), \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Historically popular, but suffers from vanishing gradients for large $|z|$.

Definition 0.2 (Hyperbolic Tangent (tanh))

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1), \quad \tanh'(z) = 1 - \tanh^2(z).$$

Zero-centered, which helps optimisation, but still saturates.

Definition 0.3 (Rectified Linear Unit (ReLU))

$$\text{ReLU}(z) = \max(0, z), \quad \text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{otherwise.} \end{cases}$$

Computationally efficient and mitigates vanishing gradients for positive inputs. May cause “dead neurons” (never activate) if learning rate is too high.

Definition 0.4 (Leaky ReLU / ELU / Swish) Variants (e.g., Leaky ReLU: $\max(\alpha z, z)$ with $\alpha \approx 0.01$) address the dying ReLU problem.

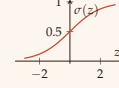


Figure 3: The sigmoid activation squashes inputs to $(0, 1)$ and is differentiable.

Multilayer Perceptron and Backpropagation

A **multilayer perceptron (MLP)** consists of an input layer, one or more hidden layers, and an output layer. Each neuron in layer ℓ receives activations $\mathbf{o}^{(\ell-1)}$ from the previous layer, computes a weighted sum plus bias, and passes it through an activation function. In vectorised form:

$$\mathbf{a}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{o}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{o}^{(\ell)} = f^{(\ell)}(\mathbf{a}^{(\ell)}),$$

where $\mathbf{o}^{(0)} = \mathbf{x}$ (the input). The output layer produces $\hat{\mathbf{y}} = \mathbf{o}^{(L)}$.

Derivation of Backpropagation

We wish to minimise a loss function $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{t})$ over the training set. For a single example, define the error at the output layer as the derivative of $\mathcal{L}$ with respect to the pre-activation $\mathbf{a}^{(L)}$:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} = \nabla_{\hat{\mathbf{y}}} \mathcal{L} \odot f^{(L)'}(\mathbf{a}^{(L)}),$$

where $\odot$ denotes elementwise multiplication. For the squared loss $\mathcal{L} = \frac{1}{2} \|\mathbf{t} - \hat{\mathbf{y}}\|^2$, we have $\nabla_{\hat{\mathbf{y}}} \mathcal{L} = \hat{\mathbf{y}} - \mathbf{t}$.

Now apply the chain rule to propagate the error backwards. For layer ℓ (with $\ell < L$), $\mathbf{a}^{(\ell)}$ influences $\mathcal{L}$ through $\mathbf{a}^{(\ell+1)} = \mathbf{W}^{(\ell+1)} \mathbf{o}^{(\ell)} + \mathbf{b}^{(\ell+1)}$ and $\mathbf{o}^{(\ell)} = f^{(\ell)}(\mathbf{a}^{(\ell)})$. Therefore,

$$\delta^{(\ell)} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}} = \left(\frac{\partial \mathbf{a}^{(\ell+1)}}{\partial \mathbf{o}^{(\ell)}} \right)^T \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell+1)}} \odot f^{(\ell)'}(\mathbf{a}^{(\ell)}) = ((\mathbf{W}^{(\ell+1)})^T \delta^{(\ell+1)}) \odot f^{(\ell)'}(\mathbf{a}^{(\ell)}).$$

The gradients with respect to weights and biases are then:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \delta^{(\ell)} (\mathbf{o}^{(\ell-1)})^T, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \delta^{(\ell)}.$$

These gradients are used in SGD (or any gradient-based optimiser) to update the network parameters. The algorithm is summarised below.

Algorithm 1: Backpropagation (stochastic / mini-batch version)

Input: Training examples $(\mathbf{x}, \mathbf{t})$, learning rate η , mini-batch size m .

Output: Trained weights and biases.

1. Initialise all $\mathbf{W}^{(\ell)}$ and $\mathbf{b}^{(\ell)}$ (e.g., using He or Xavier initialisation).
2. Repeat until convergence:
 - (a) Sample a mini-batch $\mathcal{B}$ of m examples.
 - (b) For each example in $\mathcal{B}$, perform forward propagation to compute all $\mathbf{a}^{(\ell)}$ and $\mathbf{o}^{(\ell)}$.
 - (c) Compute the output error $\delta^{(L)} = \nabla_{\mathbf{y}} \mathcal{L} \odot f^{(L)'}(\mathbf{a}^{(L)})$.
 - (d) For $\ell = L - 1$ down to 1:

$$\delta^{(\ell)} = ((\mathbf{W}^{(\ell+1)})^\top \delta^{(\ell+1)}) \odot f^{(\ell)'}(\mathbf{a}^{(\ell)}).$$

- (e) Accumulate gradients over the mini-batch:

$$\frac{\partial \mathcal{L}_{\mathcal{B}}}{\partial \mathbf{W}^{(\ell)}} \leftarrow \frac{1}{m} \sum_{i \in \mathcal{B}} \delta_i^{(\ell)} (\mathbf{o}_i^{(\ell-1)})^\top, \quad \frac{\partial \mathcal{L}_{\mathcal{B}}}{\partial \mathbf{b}^{(\ell)}} \leftarrow \frac{1}{m} \sum_{i \in \mathcal{B}} \delta_i^{(\ell)}.$$

- (f) Update parameters: $\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta \frac{\partial \mathcal{L}_{\mathcal{B}}}{\partial \mathbf{W}^{(\ell)}}$, similarly for $\mathbf{b}^{(\ell)}$.
-

Advanced Optimisation Methods

Plain SGD can be slow or sensitive to learning rate choices. Modern deep learning often uses improved optimisers:

- **Momentum:** maintains a velocity vector $\mathbf{v}_t = \gamma \mathbf{v}_{t-1} + \eta \nabla \mathcal{L}$, then updates parameters by $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{v}_t$. This dampens oscillations and accelerates convergence.
- **Adam (Adaptive Moment Estimation):** maintains per-parameter learning rates using exponentially decaying averages of gradients and squared gradients. It is robust and widely used.
- **Learning rate scheduling:** reducing the learning rate over time (e.g., step decay, cosine annealing, or warmup) helps the model fine-tune towards a minimum.

Regularisation for Neural Networks

Deep networks have many parameters and can easily overfit. Several techniques reduce overfitting:

L2 Regularisation (Weight Decay)

Add a penalty $\frac{\lambda}{2} \|\mathbf{w}\|_2^2$ to the loss. This encourages small weights and corresponds to a Gaussian prior on $\mathbf{w}$ in a Bayesian (MAP) framework.

Dropout

During training, randomly set a fraction p of neurons (and their connections) to zero at each forward pass. This prevents co-adaptation and can be interpreted as training an ensemble of thinned networks. At test time, all neurons are used but their outputs are scaled by p .

Batch Normalisation (BatchNorm)

For each mini-batch, BatchNorm normalises the pre-activations to have zero mean and unit variance, then applies a learned scale γ and shift β :

$$\mu_B = \frac{1}{m} \sum_{i \in B} a_i, \quad \sigma_B^2 = \frac{1}{m} \sum_{i \in B} (a_i - \mu_B)^2, \quad \hat{a}_i = \frac{a_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \tilde{a}_i = \gamma \hat{a}_i + \beta.$$

BatchNorm accelerates training and acts as a regulariser.

Early Stopping

Monitor the validation error during training and stop when it stops improving. This limits the effective capacity and is one of the simplest regularisation methods.

Bias–Variance Tradeoff in Neural Networks

Recall from statistical learning the bias–variance decomposition (for squared loss):

$$\mathbb{E}_{\mathcal{D}}[R(h_{\mathcal{D}})] = \underbrace{(\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}] - h^*)^2}_{\text{bias}^2} + \underbrace{\mathbb{V}_{\mathcal{D}}[h_{\mathcal{D}}]}_{\text{variance}} + \text{noise}.$$

In neural networks, the model capacity increases with the number of layers and neurons. Low capacity leads to high bias (underfitting); high capacity leads to low bias but high variance (overfitting). Regularisation and early stopping control the effective capacity, trading bias against variance. The optimal capacity is problem-dependent and is usually found via cross-validation.

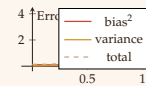


Figure 4: Typical bias–variance tradeoff: total generalisation error has a minimum at the optimal capacity.

Universal Approximation Theorem

One of the most important theoretical results for neural networks is the universal approximation theorem.

Theorem 0.1 (Cybenko, 1989) Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be any continuous, bounded, non-constant activation function (e.g., sigmoid). Then for any continuous function $f : [0, 1]^d \rightarrow \mathbb{R}$ and any $\varepsilon > 0$, there exist an integer N , vectors $\mathbf{w}_j \in \mathbb{R}^d$, scalars $v_j, b_j \in \mathbb{R}$ such that

$$\sup_{\mathbf{x} \in [0, 1]^d} \left| \sum_{j=1}^N v_j \sigma(\mathbf{w}_j \cdot \mathbf{x} + b_j) - f(\mathbf{x}) \right| < \varepsilon.$$

The theorem implies that a single hidden layer (with enough neurons) can approximate any continuous function arbitrarily well. However, it does not guarantee that such a network can be learned efficiently; the required number of neurons N may be astronomically large. Deep networks (many layers) can represent certain functions exponentially more compactly than shallow ones, which is the main motivation for deep learning.

Figure ?? illustrates the idea: approximating $\sin(x)$ with piecewise linear functions (which can be built from ReLU units). As the number of linear regions increases (i.e., as we add more hidden units), the approximation improves.

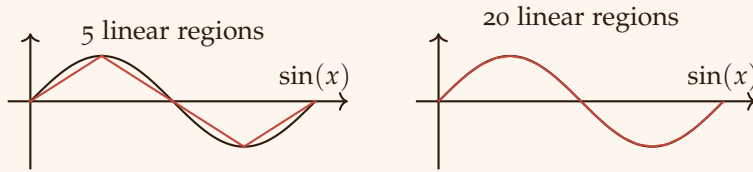


Figure 5: Approximating $\sin(x)$ with piecewise linear functions (e.g., ReLU networks). More hidden units (more linear regions) yield a better approximation, illustrating the universal approximation theorem.

Weight Initialisation

Proper initialisation is crucial for deep networks to avoid vanishing or exploding gradients. Two widely used schemes are:

- **Xavier (Glorot) initialisation:** for tanh or sigmoid, set $\text{Var}[w] = \frac{2}{n_{\text{in}} + n_{\text{out}}}$, where n_{in} and n_{out} are the numbers of input and output units of the layer.
- **He (Kaiming) initialisation:** for ReLU, set $\text{Var}[w] = \frac{2}{n_{\text{in}}}$.

These choices keep the signal variance roughly constant across layers, promoting stable gradient flow.

With these foundations – perceptron, gradient descent, back-propagation, activation functions, optimisation, regularisation,

bias–variance tradeoff, universal approximation, and initialisation
– we have a rigorous yet intuitive understanding of how neural networks learn. Subsequent chapters will build on these ideas to introduce convolutional networks, recurrent networks, and generative models.